EndCorelib user guide

End covers for the initial value problem

Bingwei Zhang Chee Yap

November 2026

Contents

1 What EndCorelib computes **2**

2 Installing **2**

2.1 What you need . . . 2
2.2 Building CAPD . . . 3
2.3 Building EndCorelib . . . 3
2.4 Checking the build . . . 4

3 A first run **4**

3.1 An example from the paper . . . 4
3.2 The same run without make . . . 5
3.3 Output levels and files . . . 5

4 A problem of your own **7**

4.1 A `.mk` file . . . 7
4.2 Writing the right-hand side . . . 9
4.3 Constants . . . 9
4.4 The form of the expression . . . 9
4.5 Mistakes in the input . . . 9

5 Following a long run **10**

5.1 One line per scaffold . . . 10
5.2 Inside one scaffold . . . 11
5.3 A long run . . . 12

6 Choosing the options **13**

6.1 The error bound . . . 13
6.2 The horizon . . . 13
6.3 The enclosure step: `stepB` . . . 14
6.4 The refinement operators: `method` . . . 14
6.5 The Taylor order . . . 15
6.6 Boundary mode . . . 15

6.7 The remaining options 16
6.8 In short 16

7 When a run stops **16**

8 Reproducing the paper **18**

9 Inside the code **19**

A The usage summary **19**

1 What EndCorelib computes

EndCorelib takes an autonomous system of ordinary differential equations

$$x' = f(x), \qquad x \in \mathbb{R}^n,$$

a box $B_0 \subset \mathbb{R}^n$ of initial values, a horizon $H > 0$ and an error bound $\varepsilon > 0$. It returns a finite set C of boxes with

$$\mathrm{End}(B_0, H) \;\subseteq\; \bigcup C \;\subseteq\; \mathrm{End}(B_0, H) + [-\varepsilon, \varepsilon]^n, \tag{1}$$

where $\mathrm{End}(B_0, H) = \{\, x(H) : x(0) \in B_0 \,\}$ is the set of states reached at time H. A set C with this property is an *ε-cover* of the end set. The left inclusion says that no solution starting in B_0 is lost. The right one holds because every box of C is at most ε wide and contains a point of the end set. All arithmetic is interval arithmetic with directed rounding, so the inclusions hold for the boxes the program prints, rounding errors included. If double precision runs out first, that is, if Refine has not reached ε when E_0 can no longer be halved, the wider box is kept, so that nothing is lost, and the program prints a note saying so.

With most validated solvers you learn the width of the result after the run. Here ε is an input: EndCorelib subdivides B_0 until every piece has an end-enclosure at most ε wide, and C is the set of these end-enclosures. The program also reports $B_1 = \mathrm{Box}(C)$, the smallest box that contains the cover. Because every box of C lies within ε of the end set, B_1 exceeds the smallest box around $\mathrm{End}(B_0, H)$ by at most ε on each side.

The cover can be much smaller than B_1. Figure 1 shows Eijgenraam's example $x' = 0$, $y' = x^2$ with $x(0) \in [-1, 1]$ and $y(0) = 0$. At $H = 1$ the end set is the arc $y = x^2$, $|x| \le 1$. Every convex set that contains the arc also contains $(0, 1)$, the midpoint of its two end points, although no solution comes near that point. The cover for $\varepsilon = 0.1$ has 40 boxes and follows the arc.

The algorithm is the one of B. Zhang and C. Yap, *End Cover for Initial Value Problem: Complete Validated Algorithm with Complexity Analysis* (arXiv:2502.00503 and arXiv:2602.00162), called *the paper* below. Section and equation numbers such as (5.5) refer to it. Section 9 shows where each part of the paper sits in the code. Everything runs in IEEE double precision.

2 Installing

2.1 What you need

- A C++17 compiler: g++ 9 or newer, or clang++ 10 or newer.

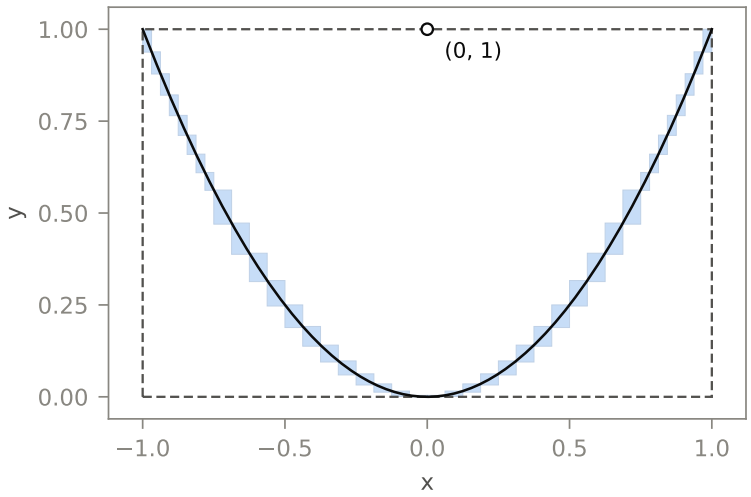


Figure 1: Eijgenraam's example at $H = 1$: the 0.1-cover (40 boxes), the end set $y = x^2$ (black) and the bounding box B_1 of the cover (dashed). The point $(0, 1)$ lies in B_1 and in every convex enclosure of the arc. Data from `make run EG=src/examples/fig1Eijgenraam.mk`.

- GNU make and bash, for the Makefile and the scripts in `src/repro/` and `src/doc/`. CMake 3.16 or newer if you prefer the CMake build.
- CAPD (`http://capd.ii.uj.edu.pl`), which supplies the interval arithmetic and the Taylor coefficients of f.
- Optional: SymEngine, which multiplies out the right-hand side before it is evaluated (Section 4.4); Python 3 with matplotlib for the plots; MATLAB for `src/tools/pendulum_ode45.m` only.

On Windows, install Cygwin with the packages `gcc-g++`, `make`, `cmake` and `git`, and type every command of this guide in a Cygwin login shell. Build EndCorelib in the same shell that built CAPD. Inside Cygwin the Windows path `C:\Users\you\CAPD` is written `/cygdrive/c/Users/you/CAPD`.

2.2 Building CAPD

Clone CAPD wherever you like and build it; you will need the path in the next step.

```
cd ~
git clone https://github.com/CAPDGroup/CAPD.git
cd CAPD && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -DCAPD_BUILD_EXAMPLES=OFF
↪ -DCAPD_BUILD_TESTS=OFF
make -j4
```

This leaves `libcapd.a` in `~/CAPD/build`. Running `make install` is optional.

2.3 Building EndCorelib

EndCorelib reads the location of CAPD from a file `config.mk` in its top directory. Copy the template and change its one active line to the absolute path of the directory you cloned CAPD into (the one that contains `capdDynSys/` and `build/`):

```
cp config.mk.example config.mk
# in config.mk:
CAPD_MASTER := /home/you/CAPD
```

`config.mk` is ignored by git, so the path stays on your machine. You can also give the path on each call, `make CAPD_MASTER=/home/you/CAPD`, or point at an installed CAPD with `make CAPD_CONFIG=/usr/local/bin/capd-config`. If `capd-config` is on your `PATH`, a plain `make` finds it.

Now run `make`. The first line it prints names the CAPD it uses and the last one is `Built EndCorelib`. If it says `CAPD not found`, check that `build/libcapd.a` exists under the directory you gave. Three companion programs are built on request: `make EndCoretraj`, a validated trajectory printer used by `make verify`; `make capdref`, the plain CAPD integrator behind the CAPD rows of the paper's Table 2; and `make EndCoreaudit`, the box-by-box check of Section 8. `make tools` builds all three.

There is also a CMake build:

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . --parallel
ctest
```

It uses `capd-config` when it finds one and otherwise downloads CAPD from GitHub and builds it first. SymEngine is used only with `-DUSE_SYMENGINE=ON`. Both builds compile with `-frounding-math`. Keep it: without it the compiler may assume round-to-nearest and break the directed rounding that every interval bound relies on.

CAPD can be built with the interval type of the filib library, which its CMake build does by default, or with its own interval type. EndCorelib uses only operations that both provide and compiles against either.

2.4 Checking the build

`make check` runs 18 short computations and compares B_1, rounded to two decimals, and the number of boxes $\#(C)$ with stored values. It takes about fifteen seconds and should end like this:

```
Boundary mode
ok    Eg1 boundary 2D  H=1                  (0.08, 1.46) +- (0.01, 0.07)  #(C)=4
ok    Eg6 boundary 3D  H=1                  (-1.74, 1.86, 0.03) +- (0.15, 0.18, 0.00)
↪  #(C)=6

18 passed, 0 failed
```

The stored values hold whether or not SymEngine was found (Section 4.4 explains why that could matter).

3 A first run

3.1 An example from the paper

The directory `src/examples/` holds the test problems of the paper's Table 1 as `.mk` files. Eg1 is the Lotka-Volterra system $x' = 2x(1-y)$, $y' = -y(1-x)$ from $B_0 = [0.9, 1.1] \times [2.9, 3.1]$, with $H = 4$ and $\varepsilon = 1$ in the file:

```
$ make run-eg1Volterra
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=1
time(ms)=278
```

```
Hull(T=4.000000)={[0.851460, 1.945561],[0.098855, 0.263816]}
B1=(1.398511, 0.181336) +- (0.547050, 0.082480)
wmax(B1)=1.094101
#(C)=2  #E0=2  max_{B in C} wmax(B)=0.997973
```

In the examples of this guide a line that starts with `$` is what was typed; the lines below it are what the program printed. Reading the output from the top:

`EndCorelib mode=…`
: the settings in force. The file says `tubedegree=0`, which selects the Euler tube, printed as `tubedegree=1`.

`f =`
: the right-hand side in the form in which it is evaluated (Section 4.4).

`B0, H, eps`
: the initial box, the horizon and ε.

`time(ms)`
: the wall-clock time of the computation in milliseconds.

`Hull(T=4.000000)`
: the box $B_1 = \mathrm{Box}(C)$, as one interval per coordinate.

`B1` the same box as midpoint ± half-width, the form of the B_1 column of the paper's Table 2.

`wmax(B1)`
: the largest side of B_1.

`#(C)`
: the number of boxes in the cover. `#E0` counts the sub-boxes of B_0 that were integrated, one for each box of C, and the last number is the largest side among the boxes of C, which is at most ε.

Here C has 2 boxes, none wider than 0.998, and B_1 is 1.094 wide in x and 0.165 wide in y.

3.2 The same run without make

`make` passes the fields of the `.mk` file to the program as positional arguments. `make show-args` prints the command instead of running it:

```
$ make show-args EG=src/examples/eg1Volterra.mk
./EndCorelib 2 0 0 0 0 2 'x' 'y' '2*x-2*x*y' '-y+x*y' 1.0 20 4.0 0 0.90000000000000002
↪  1.1000000000000001 2.8999999999999999 3.1000000000000001
```

The quotes are only there for the shell. Typed by hand, the same run is

```
$ ./EndCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 1.0 20 4 0  0.9 1.1  2.9 3.1
```

and it prints the same lines as `make run-eg1Volterra`. Table 1 lists the arguments in order. Section 6 discusses how to choose them, and the program prints a short summary when it is called without arguments (Appendix A).

3.3 Output levels and files

`output_mode` is cumulative: each level prints what the levels below it print and adds the items of Table 2. Level 2 is the usual choice. The files are written into the current directory, and every run appends one line to `out.txt`.

Here is Eg1 again with $\varepsilon = 0.1$ at level 3, followed by the first lines of `E1.txt` and the last two lines of the log:

Table 1: The command-line arguments of `EndCorelib`, in order.

argument	meaning
`output_mode`	how much is printed and written, 0 to 4 (Table 2)
`mode`	0: EndCover, cover all of B_0; 1: Boundary, cover only the boundary of B_0
`method`	the refinement operators Refine may use: 0 `tube`, 1 `bisect`, 2 `splitonly`
`stepB`	the one-step enclosure: 0 `lohner`, 1 `lohner+lognorm`, 2 `lohnerHO+lognorm`, 3 `lohnerHO`
`tubedegree`	0 or 1, both the Euler tube of the paper's Section 5.2; no other value is accepted
`n`	the dimension n
`var_1 … var_n`	the names of the variables
`f_1 … f_n`	the right-hand sides (Section 4.2)
`eps`	the error bound $\varepsilon > 0$
`order`	the Taylor order of StepB, at least 3; the paper uses 20
`T`	the horizon $H > 0$
`debug`	progress reports, 0 to 3 (Section 5)
`lo_1 hi_1 … lo_n hi_n`	the initial box B_0

Table 2: What each output level adds, and the files it writes.

level	adds
0	`time(ms)`
1	the header lines, `Hull`, `B1` and `wmax(B1)`
2	`#(C)`, `#E0` and the largest side among the boxes of C
3	`E0.txt`: the sub-boxes of B_0 that were integrated, in the order of C; `E1.txt`: the cover C; in two dimensions `convex_hull.txt`, the convex hull of C as a closed polygon
4	`E_0.txt`, `E_1.txt`: the same, plus the images of every sub-box at $t = 0.1$, 0.4 and 0.7 and, in two dimensions, the trajectories of the four corners of B_0, for plotting

```
$ ./EndCorelib 3 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.1 20 4 0  0.9 1.1  2.9 3.1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.1
time(ms)=48148
Hull(T=4.000000)={[1.201730, 1.719808],[0.165442, 0.216653]}
B1=(1.460769, 0.191047) +- (0.259039, 0.025606)
wmax(B1)=0.518079
#(C)=52  #E0=52  max_{B in C} wmax(B)=0.098052
written -> E0.txt (52 boxes)
written -> E1.txt (52 boxes)
convex hull -> convex_hull.txt (12 pts)
```

```
$ head -n 3 E1.txt
Box 0: [1.6233053559025607e+00, 1.7198082346753180e+00] x [2.0579051043088779e-01,
↪  2.1665311962151917e-01]
Box 1: [1.5884608915236910e+00, 1.6626748873045798e+00] x [2.0010058116363532e-01,
↪  2.0867734330513385e-01]
Box 2: [1.5721236857144418e+00, 1.6495902213227169e+00] x [1.9933135319574532e-01,
↪  2.0852527746167182e-01]
```

```
$ tail -n 2 out.txt
EndCover  EndCover  bisect  0  0.5  20  5.5  2  (1.09274786324,2.95578098017)  (0.53031245
↪  5911,0.231026841774)  1.06062491182  29.403199  64  {[0.9, 1.1],[2.9,
↪  3.1]}
```

```
EndCover  EndCover  tube  0  0.1  20  4  2  (1.46076897006,0.191047326955)  (0.25903926461
↪  5,0.0256057926661)  0.51807852923  48.148015  52  {[0.9, 1.1],[2.9,
↪  3.1]}
```

A box file has one box per line, `Box k: [lo, hi] x [lo, hi] …`, with 17 significant digits; line k of `E0.txt` and line k of `E1.txt` belong together. The columns of `out.txt` are separated by tabs and named in its first line. Figure 2 draws the two box files. To draw them yourself, run

```
python3 src/tools/plot_boxes.py . --save cover.png
```

in the directory that holds them.

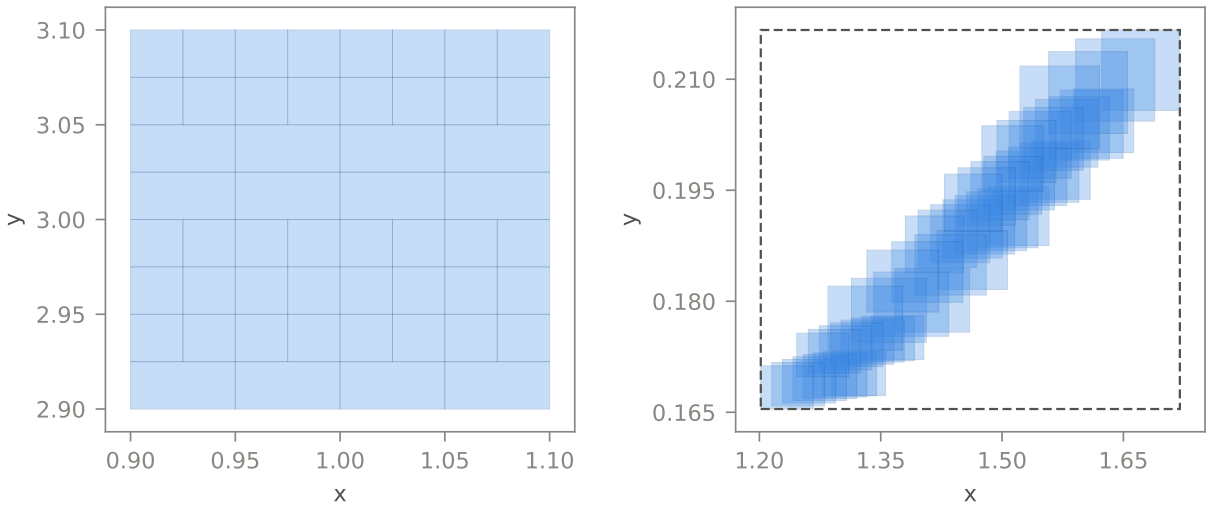


Figure 2: Eg1 with $\varepsilon = 0.1$ and $H = 4$. Left: the 52 sub-boxes of B_0 in `E0.txt`. Right: the 52 boxes of the cover C in `E1.txt` and their bounding box B_1 (dashed).

The 52 boxes of `E0.txt` tile B_0: 40 are squares of side 0.025, and 12 are 0.05 wide in x and 0.025 in y. Each is the E_0 of one scaffold, obtained from B_0 by `Split`, which halves a longest side, and line k of `E1.txt` is its end-enclosure at H.

4 A problem of your own

4.1 A `.mk` file

Take the damped pendulum $x' = y$, $y' = -\sin x - 0.2\,y$ from the box $[0.9, 1.1] \times [-0.1, 0.1]$ up to $H = 10$. The file `src/examples/dampedPendulum.mk` describes it:

```
# ========================================================================
#  A problem of your own -- the damped pendulum       (not in the paper)
#
#      x' = y
#      y' = -sin(x) - 0.2 y
#
#      B0 = Box(1,0)(0.1)         H = 10
#
#  The user guide (doc/userguide.pdf) builds this file step by step.
#      make run EG=src/examples/dampedPendulum.mk
#      make run EG=src/examples/dampedPendulum.mk eps=0.05 debug=1
# ========================================================================
n   = 2
var = x,y
```

```
ff  = y,-sin(x)-0.2*y

cen = 1.0,0.0
wid = 0.1,0.1

eps   = 0.1
order = 20
T     = 10

output_mode = 2
mode        = 0
method      = 0
stepB       = 0
tubedegree  = 0
debug       = 0
```

`var` and `ff` are comma-separated lists. `cen` and `wid` give the box as centre and half-width in each coordinate; `lohi = "0.9 1.1 -0.1 0.1"` would give it by its bounds instead. The other fields are the arguments of Table 1, and `src/examples/_template.txt` describes them all. Run the file with `make run`:

```
$ make run EG=src/examples/dampedPendulum.mk
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, -0.2*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.1
time(ms)=1989
Hull(T=10.000000)={[-0.427619, -0.304650],[0.016225, 0.139323]}
B1=(-0.366134, 0.077774) +- (0.061484, 0.061549)
wmax(B1)=0.123098
#(C)=4  #E0=4  max_{B in C} wmax(B)=0.092940
```

Any field can be set on the command line without editing the file. Here ε is halved and the progress reports of Section 5 are switched on:

```
$ make run EG=src/examples/dampedPendulum.mk eps=0.05 debug=1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, -0.2*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.05
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     2.2s      1     0  2.0e-01   4.8e-02     3   12.5%  resolved, E_0 -> 1.0e-01, 3 split
     ↪  off
     2.8s      2     3  1.0e-01   2.9e-02     3   18.8%  resolved, E_0 -> 5.0e-02, 1 split
     ↪  off
     2.8s      3     4  5.0e-02   3.3e-02     2   25.0%  resolved
     3.5s      4     2  1.0e-01   4.9e-02     2   37.5%  resolved, E_0 -> 1.0e-01, 1 split
     ↪  off
     4.2s      5     3  1.0e-01   2.8e-02     2   43.8%  resolved, E_0 -> 5.0e-02, 1 split
     ↪  off
     4.2s      6     4  5.0e-02   3.2e-02     1   50.0%  resolved
     8.5s      7     1  2.0e-01   3.1e-02     3   56.3%  resolved, E_0 -> 5.0e-02, 3 split
     ↪  off
     8.5s      8     4  5.0e-02   3.7e-02     2   62.5%  resolved
     9.4s      9     3  1.0e-01   3.1e-02     2   68.8%  resolved, E_0 -> 5.0e-02, 1 split
     ↪  off
     9.5s     10     4  5.0e-02   3.7e-02     1   75.0%  resolved
    11.9s     11     2  1.0e-01   3.1e-02     2   81.3%  resolved, E_0 -> 5.0e-02, 2 split
    ↪  off
    12.0s     12     4  5.0e-02   3.6e-02     1   87.5%  resolved
    12.8s     13     3  1.0e-01   3.2e-02     1   93.8%  resolved, E_0 -> 5.0e-02, 1 split
    ↪  off
    12.9s     14     4  5.0e-02   3.8e-02     0  100.0%  resolved
    12.9s  EndCover finished: 14 scaffolds processed, deepest level 4, #(C)=14
```

```
time(ms)=12885
Hull(T=10.000000)={[-0.409744, -0.302173],[0.024681, 0.135483]}
B1=(-0.355958, 0.080082) +- (0.053785, 0.055401)
wmax(B1)=0.110802
#(C)=14  #E0=14  max_{B in C} wmax(B)=0.049363
```

4.2 Writing the right-hand side

An f_i may use `+ - * /`, parentheses, powers `x^3`, and the functions `sin`, `cos`, `exp`, `log`, `sqrt`, `sqr`, `atan`, `asin` and `acos`. A number directly followed by a name or an opening parenthesis is multiplied by it, so `2x` is `2*x` and `(x+1)(y-1)` is `(x+1)*(y-1)`; digits inside a name, as in `x1` or `k2a`, belong to the name. In a `.mk` file the commas separate the f_i, so an f_i cannot contain one. On the command line each f_i is a separate argument: quote it when it contains parentheses or other characters the shell would interpret,

```
$ ./EndCorelib 1 0 0 0 0 2 x y y "-sin(x)-2e-1*y" 0.1 20 10 0  0.9 1.1  -0.1 0.1
```

4.3 Constants

A decimal constant such as `0.2` is read as the double nearest to it and enters the computation as an interval of width zero. The cover is then rigorous for the system whose coefficient is that double, which differs from 0.2 by about 10^{-17}. If the coefficient is meant to be exactly $1/5$, write `1/5`: a quotient is evaluated in interval arithmetic and its enclosure contains the exact value, and the same holds for `8/3` in the Lorenz example. Constants in scientific notation are turned into such quotients, `2e-1` into `(2/10^1)`. The header of the run above shows the result:

```
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, (-1/5)*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.1
time(ms)=3041
Hull(T=10.000000)={[-0.427619, -0.304650],[0.016225, 0.139323]}
B1=(-0.366134, 0.077774) +- (0.061484, 0.061549)
wmax(B1)=0.123098
```

4.4 The form of the expression

Interval evaluation depends on how an expression is written: $x(1-x)$ and $x-x^2$ are the same function but give different enclosures over a box. If SymEngine was found when EndCorelib was built, every f_i is first multiplied out into a sum of monomials, so that `(1-x^2)*y-x` becomes `-x+y-x^2*y`. Without SymEngine it is evaluated as written. The `f =` line of the output shows the form that was used.

On Lorenz (Eg5) the difference shows in the work. Run to $H = 5$ with $\varepsilon = 1$ and `stepB=0`, written as `10*(y-x), x*(28-z)-y, x*y-8*z/3` and evaluated as written, EndCover split B_0 down to depth 7, processed 141 scaffolds and returned 103 boxes. Multiplied out, it stopped at depth 6 after 95 scaffolds and 64 boxes. The files in `src/examples/` and the scripts in `src/repro/` therefore spell out the multiplied-out form. That is also the form the numbers of the paper were computed from, so these files give the same results in builds with and without SymEngine.

4.5 Mistakes in the input

A wrong argument count, a negative ε or a box with `lo > hi` stops the program with `Argument error` and the usage summary. A right-hand side that cannot be read is reported with the map string the parser was given; here `Y` should have been `y`:

```
$ ./EndCorelib 1 0 0 0 0 2 x y y "-sin(x)-0.2*Y" 0.1 20 10 0  0.9 1.1  -0.1 0.1
Cannot read the right-hand side var:x,y;fun:y,-0.2*Y-sin(x);
undefined symbol 'Y' in the expression!
If this is a numerical constant in scientific notation please replace it by a parameter.
```

Both cases end with exit status 2.

5 Following a long run

A run can take hours, and with `debug=0` it prints nothing until it is finished. The `debug` argument switches on progress reports:

`debug`	reports
0	none
1	one line for every scaffold that EndCover has taken from its queue
2	also one line for every scaffold laid out to the horizon and for every pass of Refine
3	also one line for every stage appended by Extend, and the containment self-checks, which should never fire

Each line starts with the wall-clock time since the program started. The reports go to standard error and the results to standard output, so `> result.txt` still captures the results alone, and `2> progress.log` keeps the reports in a file you can follow with `tail -f progress.log`.

5.1 One line per scaffold

Eg1 with $\varepsilon = 0.5$ at `debug=1`:

```
$ ./EndCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 1  0.9 1.1  2.9 3.1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.8s      1     0  2.0e-01   4.1e-01     2   25.0%  resolved, E_0 -> 1.0e-01, 2 split
     ↪  off
     1.0s      2     2  1.0e-01   2.4e-01     2   37.5%  resolved, E_0 -> 1.0e-01, 1 split
     ↪  off
     1.1s      3     3  1.0e-01   2.6e-01     1   50.0%  resolved
     1.3s      4     1  2.0e-01   3.9e-01     1   75.0%  resolved, E_0 -> 1.0e-01, 1 split
     ↪  off
     1.4s      5     2  1.0e-01   4.6e-01     0  100.0%  resolved
     1.4s  EndCover finished: 5 scaffolds processed, deepest level 3, #(C)=5
time(ms)=1407
Hull(T=4.000000)={[1.093289, 1.784663],[0.146599, 0.229647]}
B1=(1.438976, 0.188123) +- (0.345687, 0.041524)
wmax(B1)=0.691374
#(C)=5  #E0=5  max_{B in C} wmax(B)=0.458567
```

Each scaffold taken from the queue gets one line, written when it is finished:

`run`
: the number of the scaffold, in the order processed.

`depth`
: how often B_0 was split to obtain its initial box B; B_0 itself has depth 0. `Split` halves a longest side, so in n dimensions $w_{\max}(B)$ halves every n levels.

`wmax(E_m)`
: the width of its end-enclosure at H.

`queue`
: the scaffolds still waiting.

`done`
: the share of B_0, by volume, that the cover already covers.

`outcome`
: `resolved`: the scaffold reached H with an end-enclosure at most ε wide, which goes into C. `resolved, E_0 -> w, k split off`: on the way Refine called `Split` k times, so E_0 ended at width w, and the k halves split off went into the queue as new scaffolds. `split:` the scaffold could not be carried to the horizon, for the reason given, and its initial box was split in two (Section 7).

EndCover always takes the scaffold queued last, so it works depth first. In this run Refine split E_0 twice in run 1, first along x and then along y, and the two halves it split off became runs 4 and 2. Run 2 split once more, and its half became run 3; run 4 split once, and its half became run 5. The five end-enclosures form C, and the five final E_0 tile B_0: three squares of side 0.1 and two rectangles of 0.05 by 0.1.

5.2 Inside one scaffold

At `debug=2` each scaffold also shows how it was built and refined:

```
$ ./EndCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 2  0.9 1.1  2.9 3.1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.1s      scaffold laid out to t=4: 55 stages, wmax(E_0)=2.0e-01 wmax(E_m)=6.3e+00
     0.3s      Refine pass 1: wmax(E_m)=8.0e-01 wmax(E_0)=2.0e-01 mu_max=3.39 m=55
     ↪  bisected=55 frozen=0 splits=1
     0.8s      Refine pass 2: wmax(E_m)=4.1e-01 wmax(E_0)=1.0e-01 mu_max=3.04 m=55
     ↪  bisected=55 frozen=0 splits=1
     0.8s      1     0  2.0e-01   4.1e-01     2   25.0%  resolved, E_0 -> 1.0e-01, 2 split
     ↪  off
     0.9s      scaffold laid out to t=4: 44 stages, wmax(E_0)=1.0e-01 wmax(E_m)=5.2e-01
     1.1s      Refine pass 1: wmax(E_m)=2.4e-01 wmax(E_0)=1.0e-01 mu_max=2.87 m=44
     ↪  bisected=44 frozen=0 splits=1
     1.1s      2     2  1.0e-01   2.4e-01     2   37.5%  resolved, E_0 -> 1.0e-01, 1 split
     ↪  off
     1.2s      scaffold laid out to t=4: 42 stages, wmax(E_0)=1.0e-01 wmax(E_m)=2.6e-01
     1.2s      3     3  1.0e-01   2.6e-01     1   50.0%  resolved
```

`scaffold laid out` means that Extend has reached $t = H$ in m stages; the widths of E_0 and of the end-enclosure E_m follow. Each `Refine pass` line reports one pass of the main loop of Refine: the new widths of E_m and E_0; `mu_max`, the logarithmic-norm bound that enters the tests (5.4) and (5.5); the number of stages m; how many stages `Bisect` refined in this pass; how many stages are frozen by (5.4); and how many times `Split` halved E_0. For run 1, Extend reached $H = 4$ in 55 stages with an end-enclosure 6.3 wide. Each of the two passes of Refine bisected all 55 stages and split once, which brought E_m down to 0.80 and then to 0.41, below $\varepsilon = 0.5$. Each half that `Split` gives away starts a scaffold of its own, laid out from $t = 0$: run 2, the half split off in the second pass, reached H in 44 stages with an end-enclosure 0.52 wide, and one pass of Refine brought it to 0.24. Run 3 was within ε as soon as it reached H, so Refine had nothing to do.

At `debug=3` every stage appears as Extend appends it, with its end time t_i, its length h and its end-enclosure E_i:

```
$ ./EndCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 3  0.9 1.1  2.9 3.1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.0s        stage 1: t=0.0683917 h=6.8e-02 wmax(E)=2.4e-01 E={[0.670312,
     ↪  0.852826],[2.85598, 3.09299]}
     0.0s        stage 2: t=0.109407 h=4.1e-02 wmax(E)=2.5e-01 E={[0.563562,
     ↪  0.733611],[2.81151, 3.06553]}
     0.0s        stage 3: t=0.158235 h=4.9e-02 wmax(E)=2.7e-01 E={[0.459635,
     ↪  0.616735],[2.74636, 3.01459]}
     0.0s        stage 4: t=0.203157 h=4.5e-02 wmax(E)=2.8e-01 E={[0.381751,
     ↪  0.529877],[2.67632, 2.95557]}
     0.0s        stage 5: t=0.246126 h=4.3e-02 wmax(E)=2.9e-01 E={[0.320527,
     ↪  0.461663],[2.60361, 2.89022]}
     0.0s        stage 6: t=0.296907 h=5.1e-02 wmax(E)=2.9e-01 E={[0.262497,
     ↪  0.395432],[2.51361, 2.80423]}
     0.0s        stage 7: t=0.343782 h=4.7e-02 wmax(E)=2.9e-01 E={[0.219436,
     ↪  0.34597],[2.42739, 2.72002]}
```

Short stages mark the parts of $[0, H]$ where StepA or the error control of StepB allowed only small steps.

5.3 A long run

Lorenz (Eg5) to $H = 5$ with $\varepsilon = 1$, from the `.mk` file, begins

```
$ make run EG=src/examples/eg5Lorenz.mk T=5 debug=1
EndCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=3
         f  = (-10*x+10*y, 28*x-y-x*z, (-8/3)*z+x*y)
         B0 = {[14.999, 15.001],[14.999, 15.001],[35.999, 36.001]}   H=5   eps=1
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     2.2s      1     0  2.0e-03         -     2    0.0%  split: Extend: StepB cannot
     ↪  advance at t=3.79381 (H=5)
     4.2s      2     1  2.0e-03         -     3    0.0%  split: Extend: StepB cannot
     ↪  advance at t=3.94876 (H=5)
     6.5s      3     2  2.0e-03         -     4    0.0%  split: Extend: StepB cannot
     ↪  advance at t=4.43131 (H=5)
     8.8s      4     3  1.0e-03         -     5    0.0%  split: Extend: StepB cannot
     ↪  advance at t=4.64673 (H=5)
    11.3s      5     4  1.0e-03         -     6    0.0%  split: Extend: StepB cannot
    ↪  advance at t=4.69029 (H=5)
```

and ends

```
   535.0s     91     4  1.0e-03         -     2   93.8%  split: Extend: StepB cannot
   ↪  advance at t=4.68567 (H=5)
   548.4s     92     5  1.0e-03   3.5e-01     2   95.3%  resolved, E_0 -> 5.0e-04, 1 split
   ↪  off
   550.1s     93     6  5.0e-04   8.4e-01     1   96.9%  resolved
   564.9s     94     5  1.0e-03   3.5e-01     1   98.4%  resolved, E_0 -> 5.0e-04, 1 split
   ↪  off
   566.6s     95     6  5.0e-04   8.5e-01     0  100.0%  resolved
   566.6s  EndCover finished: 95 scaffolds processed, deepest level 6, #(C)=64
time(ms)=566619
Hull(T=5.000000)={[1.016192, 1.704451],[1.850605, 2.943578],[16.076084, 16.992449]}
B1=(1.360322, 2.397091, 16.534266) +- (0.344130, 0.546487, 0.458183)
wmax(B1)=1.092973
#(C)=64  #E0=64  max_{B in C} wmax(B)=0.941293
```

The scaffold of B_0 could not be carried past $t = 3.79$ and those of its halves past $t = 3.95$; the boxes of depths 2, 3 and 4 got to $t = 4.43$, 4.65 and 4.69. The 32 boxes of depth 5 reached $H = 5$, Refine split each of them once more, and the 64 end-enclosures, all from boxes of depth 6, form C; a scaffold that was split in two contributes nothing. The `done` column moves by $1/64 \approx 1.6\%$ with each resolved scaffold, so after a few minutes of a long run it tells you roughly how much longer to wait.

6 Choosing the options

The inputs that decide what you get are ε and H. The other options decide how the work is done, and on the problems of the paper most of them change the time much more than the result. Every run below starts from the settings of the example files (`mode=0`, `method=0`, `stepB=0`, `tubedegree=0`, `order=20`), changes one option, and runs the problem to the longer horizon of the paper's Table 2. The runs come from `src/doc/compare_options.sh`, five or six at a time on an Intel Core i7-13700HX, and `src/doc/make_tables.py` turned them into the tables, so both can be regenerated. Times are wall-clock seconds. A run that was still going after ten minutes was stopped; its cell shows >600 and has no result. Running several at a time makes the times less precise, so read only the larger differences in the tables, and compare times within a table rather than with your own machine. The last table was measured two runs at a time.

6.1 The error bound

Table 3: Time and number of boxes as ε shrinks (all other options at their defaults).

		$\varepsilon = 1$		$\varepsilon = 0.5$		$\varepsilon = 0.1$	
problem	H	s	$\#(C)$	s	$\#(C)$	s	$\#(C)$
Eg1 Lotka-Volterra	5.5	24	48	30	64	360	274
Eg2 Van der Pol	2	1.2	4	2.0	6	41	51
Eg3 quadratic	4	0.9	8	1.7	13	61	143
Eg4 FitzHugh-Nagumo	4	0.7	2	0.8	2	1.8	6
Eg5 Lorenz	4	55	8	74	8	>600	–
Eg6 Rössler	4	0.2	1	0.3	1	>600	–

Going from $\varepsilon = 1$ to $\varepsilon = 0.5$ increased the number of boxes by at most 60%, and left it unchanged on FitzHugh-Nagumo, Lorenz and Rössler (Table 3). From 0.5 to 0.1 the number of boxes grew by a factor of 3 to 11 where the runs finished, and the time by a factor of up to 35; Lorenz and Rössler did not finish within ten minutes, although Rössler took 537 seconds when it ran two at a time (Table 8). Rössler is also the extreme case at the other end: without any subdivision its end-enclosure is 0.495 wide, so $\varepsilon = 0.5$ takes 0.3 seconds. Every split halves one side of a box, so the cost depends on how far ε lies below the width of the undivided end-enclosure. Start with a generous ε, look at $\#(C)$ and the time, and lower it in steps.

6.2 The horizon

The horizon sets how far the end set has spread by the time the cover is taken. For Lotka-Volterra with $\varepsilon = 0.1$ the cover has 52 boxes at $H = 4$ (Section 3) and 274 at $H = 5.5$. For a chaotic system the horizon also decides whether a run can finish at all, because every level of splitting doubles the work and gains only a fixed amount of time; Section 7 shows how to read that amount off the progress report.

6.3 The enclosure step: `stepB`

Table 4: The four StepB variants: time and $w_{\max}(B_1)$.

			time (s) for `stepB`				$w_{\max}(B_1)$ for `stepB`			
problem	H	ε	0	1	2	3	0	1	2	3
Eg1 Lotka-Volterra	5.5	0.5	30	55	39	13	1.061	1.061	0.976	0.976
Eg2 Van der Pol	2	0.5	2.0	3.4	2.6	0.8	0.705	0.705	0.693	0.693
Eg3 quadratic	4	0.5	1.7	3.4	3.7	1.2	0.944	0.944	0.942	0.942
Eg4 FitzHugh-Nagumo	4	0.5	0.8	1.2	0.1	0.1	0.214	0.214	0.207	0.207
Eg5 Lorenz	4	0.5	74	124	240	51	0.447	0.447	0.354	0.354
Eg6 Rössler	4	0.5	0.3	0.5	0.2	0.1	0.495	0.495	0.484	0.484
Eg1 Lotka-Volterra	5.5	0.1	360	>600	569	151	0.857	–	0.855	0.855
Eg2 Van der Pol	2	0.1	41	73	56	16	0.609	0.609	0.609	0.609
Eg3 quadratic	4	0.1	61	128	67	21	0.801	0.801	0.801	0.801
Eg4 FitzHugh-Nagumo	4	0.1	1.8	3.2	1.6	0.6	0.164	0.164	0.167	0.167
Eg5 Lorenz	4	0.1	>600	>600	>600	>600	–	–	–	–
Eg6 Rössler	4	0.1	>600	>600	548	130	–	–	0.464	0.464
Eg5 Lorenz	5	1	450	>600	412	111	1.093	–	0.923	0.923

Variants 1 and 2 intersect each enclosure with the ball of the paper's Lemma 2.1, and variants 2 and 3 use Lohner's method with a higher-order a priori enclosure. In every row of Table 4 in which both finished, variant 1 returned the cover of variant 0, and variant 3 that of variant 2, with the same number of boxes and the same B_1: the ball narrowed nothing. It did cost time, a factor of 1.5 to 2.1 for variant 1 over variant 0 and of 2 to 5 for variant 2 over variant 3.

That leaves variants 0 and 3. Variant 3 was the faster one in every row in which both finished, by a factor between 1.4 and 12, and it also finished Rössler at $\varepsilon = 0.1$. Its B_1 was narrower in every row but one, by up to 21% (Lorenz at $\varepsilon = 0.5$), and 1.6% wider on FitzHugh-Nagumo at $\varepsilon = 0.1$. On Lorenz to $H = 5$ with $\varepsilon = 1$ variant 3 took 111 seconds and variant 0 took 450, for the same 64 boxes, and the hull of variant 3 was 16% narrower. Variant 0 is the setting of the paper's experiments and of the example files; for a new problem, try variant 3 first.

6.4 The refinement operators: `method`

Table 5: The three refinement strategies: time and $w_{\max}(B_1)$.

			time (s) for `method`			$w_{\max}(B_1)$ for `method`		
problem	H	ε	0	1	2	0	1	2
Eg1 Lotka-Volterra	5.5	0.5	30	29	41	1.061	1.061	1.074
Eg2 Van der Pol	2	0.5	2.0	1.4	0.6	0.705	0.705	0.724
Eg3 quadratic	4	0.5	1.7	1.1	0.4	0.944	0.944	0.977
Eg4 FitzHugh-Nagumo	4	0.5	0.8	0.8	0.5	0.214	0.214	0.214
Eg5 Lorenz	4	0.5	74	31	30	0.447	0.465	0.458
Eg6 Rössler	4	0.5	0.3	0.2	0.2	0.495	0.495	0.495
Eg1 Lotka-Volterra	5.5	0.1	360	151	131	0.857	0.858	0.867
Eg2 Van der Pol	2	0.1	41	16	4.7	0.609	0.609	0.611
Eg3 quadratic	4	0.1	61	18	3.6	0.801	0.801	0.803
Eg4 FitzHugh-Nagumo	4	0.1	1.8	2.2	1.0	0.164	0.164	0.164
Eg5 Lorenz	4	0.1	>600	>600	215	–	–	0.138
Eg6 Rössler	4	0.1	>600	415	79	–	0.464	0.464

Method 2 was the fastest in every row of Table 5 but Lotka-Volterra at $\varepsilon = 0.5$, by up to a factor of 17 at $\varepsilon = 0.1$, and the only strategy that finished Lorenz at $\varepsilon = 0.1$. It paid for this with a few more boxes in some rows and a B_1 up to 3.5% wider. Method 1, Bisect without the tube, was faster than method 0 in all but one row, by up to a factor of 3.4, and it finished Rössler at $\varepsilon = 0.1$, which method 0 did not; its B_1 was that of method 0 to within 0.2%, except on Lorenz at $\varepsilon = 0.5$, where it was 4% wider. Method 0 is Refine as the paper states it, and the setting of its experiments; use method 2 when time matters.

6.5 The Taylor order

Table 6: The Taylor order of StepB: time and $w_{\max}(B_1)$.

			time (s) for `order`				$w_{\max}(B_1)$ for `order`			
problem	H	ε	10	15	20	30	10	15	20	30
Eg1 Lotka-Volterra	5.5	0.5	69	41	30	40	1.037	1.008	1.061	1.136
Eg5 Lorenz	4	0.5	151	129	74	324	0.395	0.368	0.447	0.350
Eg6 Rössler	4	0.5	0.6	0.3	0.3	1.6	0.483	0.489	0.495	0.489

The order changes the step sizes, and with them the whole run, in ways that are hard to predict (Table 6). Order 20, the paper's value and the one in the example files, was the fastest in every row. The other orders often returned a narrower hull: on Lorenz orders 10, 15 and 30 gave one 12% to 22% narrower, at 1.7 to 4.4 times the time; on Lotka-Volterra order 15 one 5% narrower, at 1.3 times the time, while order 30 gave one 7% wider; and on Rössler every other order one 1% to 2.5% narrower. Keep 20 unless the hull matters more than the time.

6.6 Boundary mode

Table 7: EndCover (`mode=0`) and Boundary mode (`mode=1`) at $\varepsilon = 0.1$.

		`mode=0` (EndCover)			`mode=1` (Boundary)		
problem	H	s	$\#(C)$	$w_{\max}(B_1)$	s	$\#(C)$	$w_{\max}(B_1)$
Eg1 Lotka-Volterra	5.5	360	274	0.857	37	41	0.850
Eg2 Van der Pol	2	41	51	0.609	19	19	0.612
Eg3 quadratic	4	61	143	0.801	7.1	27	0.805
Eg4 FitzHugh-Nagumo	4	1.8	6	0.164	1.6	7	0.161
Eg5 Lorenz	4	>600	–	–	>600	–	–
Eg6 Rössler	4	>600	–	–	325	104	0.465

Boundary mode covers only the boundary of B_0: it runs EndCover on each of the four edges in two dimensions, the $2n$ faces in n dimensions. The flow maps the boundary of B_0 onto the boundary of the end set, so the bounding box B_1 of this cover encloses $\mathrm{End}(B_0, H)$ as well; the boxes themselves do not cover the interior of the end set. In Table 7 boundary mode was faster wherever both modes finished, by a factor of up to 10, and it finished Rössler in 325 seconds, where EndCover was stopped after ten minutes. It needed fewer boxes except on FitzHugh-Nagumo, and its B_1 was within 0.9% of the one EndCover found, and 2% narrower on FitzHugh-Nagumo. On Lorenz neither mode finished. Use it when B_1 is all you need.

6.7 The remaining options

`tubedegree` has one meaningful value: 0 and 1 both select the Euler tube of the paper's Section 5.2, and the program refuses anything else. `output_mode=2` is right for most runs; level 3 writes the box files and level 4 also computes, for every sub-box of B_0, its images at $t = 0.1$, 0.4 and 0.7 (those not beyond H), which costs one more integration from each sub-box for each of these times. `debug=1` prints a line per scaffold and costs nothing you would notice, so it is worth switching on for anything that runs longer than a minute; levels 2 and 3 write far more and are meant for looking into one scaffold.

6.8 In short

Table 8: The settings of the example files against `method=2` with `stepB=3`.

			the example files			`method=2 stepB=3`		
problem	H	ε	s	$\#(C)$	$w_{\max}(B_1)$	s	$\#(C)$	$w_{\max}(B_1)$
Eg1 Lotka-Volterra	5.5	0.1	234	274	0.857	22	253	0.861
Eg2 Van der Pol	2	0.1	27	51	0.609	1.8	47	0.610
Eg3 quadratic	4	0.1	44	143	0.801	2.3	144	0.803
Eg4 FitzHugh-Nagumo	4	0.1	1.5	6	0.164	0.4	4	0.167
Eg5 Lorenz	4	0.1	>600	–	–	40	64	0.135
Eg6 Rössler	4	0.1	537	176	0.464	16	176	0.464
Eg5 Lorenz	5	1	339	64	1.093	52	64	0.953

- Choose ε and H first. They decide the result and most of the cost. Lower ε in steps, and for a chaotic problem read the reachable horizon off the progress report (Section 7).
- Switch on `debug=1` for anything that runs longer than a minute.
- The example files use the settings of the paper's experiments, `method=0`, `stepB=0` and `order=20`; keep them when you compare with the paper's tables.
- For speed, use `method=2` and `stepB=3`. Run together (Table 8), they finished every problem within a minute, also Lorenz at $\varepsilon = 0.1$, which the settings of the example files did not finish within ten minutes, and were 4 to 33 times faster on the others. Their B_1 was at most 1.7% wider than the one of the example settings, and 13% narrower on Lorenz to $H = 5$ at $\varepsilon = 1$.
- Write the right-hand side multiplied out, or build with SymEngine (Section 4.4).
- Use `mode=1` only when B_1 is all you need.

7 When a run stops

When a scaffold cannot be carried to H, typically because StepB cannot find a step for an enclosure that has grown too wide, EndCover splits its initial box in two along a longest side and goes on with the halves; with `debug=1` such a scaffold shows up as `split:` followed by the reason and the time reached. A smaller box has a narrower enclosure, so the halves usually get further. The run stops only when double precision is exhausted, that is, when such a box can no longer be halved.

message	meaning and remedy
`Argument error:` … followed by the usage summary, exit status 2	an argument is missing or out of range; the message says which
`Cannot read the right-hand side …`, exit status 2	the parser rejected an f_i (Section 4.2)
`[EndCover stopped] double precision exhausted … cannot be halved in double precision. Try a smaller T.`, exit status 3	a scaffold could not be carried to H and its initial box cannot be halved in double precision. Lower T; see the example below
`[EndCover stopped] EndCover: exceeded maxSplits …`, exit status 3	more than two million splits; raise ε or lower T
`[Boundary stopped] …`, exit status 3	the same for `mode=1`

With chaotic systems double precision is what runs out. Lorenz from the box of Eg5 to $H = 100$, with `stepB=3` and `debug=1`, starts

```
$ make run EG=src/examples/eg5Lorenz.mk T=100 stepB=3 debug=1
EndCorelib  mode=EndCover  method=tube  stepB=lohnerHO  tubedegree=1  order=20  n=3
         f  = (-10*x+10*y, 28*x-y-x*z, (-8/3)*z+x*y)
         B0 = {[14.999, 15.001],[14.999, 15.001],[35.999, 36.001]}   H=100   eps=1
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.5s      1     0  2.0e-03         -     2    0.0%  split: Extend: StepB cannot
     ↪  advance at t=3.83076 (H=100)
     1.1s      2     1  2.0e-03         -     3    0.0%  split: Extend: StepB cannot
     ↪  advance at t=4.01253 (H=100)
     1.8s      3     2  2.0e-03         -     4    0.0%  split: Extend: StepB cannot
     ↪  advance at t=4.34386 (H=100)
```

and ends

```
   226.7s    119   118  7.1e-15         -   120    0.0%  split: Extend: StepB cannot
   ↪  advance at t=26.4064 (H=100)
   229.6s    120   119  7.1e-15         -   121    0.0%  split: Extend: StepB cannot
   ↪  advance at t=26.413 (H=100)
   232.5s    121   120  7.1e-15         -   122    0.0%  split: Extend: StepB cannot
   ↪  advance at t=26.4072 (H=100)
   235.2s    122   121  7.1e-15         -   121    0.0%  stopped: Extend: StepB cannot
   ↪  advance at t=26.4064 (H=100)
[EndCover stopped] double precision exhausted
Extend: StepB cannot advance at t=26.4064 (H=100)
The sub-box of B0 on which this happened (depth 121 of the subdivision) cannot be halved in
↪  double precision.
Try a smaller T.
make: *** [Makefile:194: run] Error 3
```

No scaffold reaches H, so EndCover splits each box and goes on with its lower half, one level deeper each time; three levels halve every side once. B_0 got to $t = 3.83$ and the box of depth 3 to $t = 4.68$. Up to depth 90 every three levels bought 0.72 time units on average, close to $\ln 2/0.906 \approx 0.765$, where 0.906 is the largest Lyapunov exponent of the Lorenz system; the box of depth 60, a million times narrower than B_0, got to $t = 16.86$. From depth 96 on the times stay between $t = 26.40$ and 26.78: at order 20 and in double precision the enclosure of a single solution cannot be carried further, whatever ε is. At depth 121 no side of the box can be halved in double precision, and the program stops, after about four minutes.

Long before that limit the cost becomes the problem, and the progress report lets you estimate it. Write t_d for the time that boxes of depth d reach; in this run $t_0 = 3.83$, $t_3 = 4.68$, $t_6 = 5.46$, $t_9 = 6.17$ and $t_{12} = 6.86$. Covering B_0 to the horizon H needs boxes of a depth d with $t_d \geq H$, and there are 2^d of them. For Lorenz that means depth 6 and 64 boxes for $H = 5$ (the run of Section 5), presumably 512 for $H = 6$, and eight times more for every further 0.72

time units. A run whose scaffolds stop at about the same time while the depth keeps growing has met the precision limit; lower H.

8 Reproducing the paper

The tables and figures of the paper are regenerated by make targets in the top directory:

```
make table2      # the Ours(eps) rows of Table 2
make compare     # that run next to the printed table, cell by cell
make figure1     # the data of Figure 1
make figure4     # the data of Figure 4
make verify      # sample points of B0 checked against the covers
make audit       # every box of a cover checked against its own sub-box
```

`make table2` writes `repro/out/table2.txt` and `repro/out/table2.tex`. The variables `EPS`, `METHOD` and `STEPB` change its settings, and `bash src/repro/table2.sh eg1 eg5` restricts it to some of the problems. `make compare` reads `repro/table2_paper.txt`, which holds the table as printed, and marks every cell that differs. Timings depend on the machine; the paper's were taken on an Intel Core i7-13700HX with 16 GB of memory, and so were the timings in this guide.

`make figure1` computes Eijgenraam's example for $\varepsilon = 0.3$, 0.1 and 0.02 and the pendulum trajectories of the top row; `make figure4` writes `E0.txt` and `E1.txt` for each problem into `repro/out/fig4/`. The scripts `src/tools/plot_eijgenraam.py`, `src/tools/plot_pendulum.py` and `src/tools/plot_boxes.py` draw them; the README gives the exact commands.

`make verify` takes a grid of sample points of B_0 for each problem, encloses the solution through each point with `EndCoretraj`, and checks that every such enclosure lies inside some box of C. This tests the left inclusion of (1), that nothing is lost. It does not test the right one, so a box of C that is far from the end set would pass. `make audit` checks the cover box by box: for every line k of `E0.txt` it encloses the solutions through the centre and the corners of that sub-box and checks that each enclosure meets line k of `E1.txt`. A box that fails cannot be an end-enclosure of its sub-box:

```
$ make audit
eps=0.1 stepB=0 order=20
eij  H=1   method=0  ok    40 boxes, 200 images: 0 boxes miss an image of their sub-box
eij  H=1   method=1  ok    40 boxes, 200 images: 0 boxes miss an image of their sub-box
eij  H=1   method=2  ok    40 boxes, 200 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=0  ok    52 boxes, 260 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=1  ok    53 boxes, 265 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=2  ok    56 boxes, 280 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=0  ok    29 boxes, 145 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=1  ok    29 boxes, 145 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=2  ok    31 boxes, 155 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=0  ok    16 boxes, 80 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=1  ok    16 boxes, 80 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=2  ok    16 boxes, 80 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=0  ok    8 boxes, 40 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=1  ok    8 boxes, 40 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=2  ok    8 boxes, 40 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=0  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=1  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=2  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=0  ok    48 boxes, 432 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=1  ok    48 boxes, 432 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=2  ok    48 boxes, 432 images: 0 boxes miss an image of their sub-box
```

9 Inside the code

The program is a set of headers and one main file. Table 9 says where each part of the paper is implemented; `CODE_SUMMARY.md` describes every function.

Table 9: The source files and the parts of the paper they implement.

file	contents	paper
`types.h`	boxes, $w_{\max}$, midpoints, the logarithmic norm, `Scaffold`, `MiniScaffold`	2.1, 2.4, 3.2, 5.1
`stepAB.h`	StepA, the four StepB variants, h_{euler}, and `SolutionChain`, which carries one Lohner doubleton along a scaffold	4.1, 4.2, (2.2), (2.6)
`extend.h`	Extend: append one stage; `ExtendToHorizon`, `RebuildScaffold`	4, 3.3
`refine.h`	Refine and its operators: `Evaluate`, `Bisect`, `EulerTube`, `Split`, and the tests (5.4) to (5.6)	5
`endcover.h`	EndCover: the queue of scaffolds	3.3
`boundary.h`	`runScaffold`, which runs Extend and Refine for one scaffold, and the Boundary mode	3.3
`progress.h`	the progress reports of Section 5	
`symparse.h`	clean-up of the right-hand side (Section 4.2)	
`EndCorelib.cpp`	command line, output levels, output files	7

A run of `mode=0` goes through the files in this order. `main` reads the arguments and calls `EndCover` with B_0. EndCover keeps a queue of scaffolds, starting with $\text{Init}(B_0)$, and calls `runScaffold` for each. `runScaffold` lays out a new scaffold to $t = H$ with `ExtendToHorizon`, one stage per call of Extend, and then calls Refine. Each stage takes one step of StepB along a `SolutionChain`, whose length StepA bounds. Refine uses `Bisect`, `EulerTube` and `Split` as `method` allows and walks the chain again with `Evaluate` after each change. `Split` halves E_0 along a longest side: the scaffold goes on with one half, and the other half goes into the queue as the initial box of a new scaffold. When Refine returns, the end-enclosure of the scaffold goes into C. Names in the code follow the paper: `S.E`, `S.F`, `S.G` and `S.t` are the components of a scaffold $S = (t, E, F, G)$, `eps0` is ε_0, and the tests (5.4), (5.5) and (5.6) are `StageFreeze`, `NeedSplit` and `CanTerminate`.

A The usage summary

```
$ ./EndCorelib
Argument error: too few arguments

Usage: EndCorelib output_mode mode method stepB tubedegree n \
               var1 ... varn  f1 ... fn \
               eps order T debug  lo1 hi1 ... lon hin

  output_mode 0=time 1=+hull 2=+counts 3=+E0/E1 files 4=+plot files
  mode        0=EndCover 1=Boundary
  method      0=tube (default) 1=bisect 2=splitonly
  stepB       0=lohner 1=lohner+lognorm 2=lohnerHO+lognorm 3=lohnerHO
  tubedegree  0 or 1: the Euler tube (the only one available)
  debug       0=silent 1=per sub-box 2=+Refine passes 3=+stages (on stderr)
```